Dv Accumulation

Component Design Document

1 Description

Delta-V accumulation. Integrates the majority voted body frame acceleration that gravity does not cause, one sample per tick over the control period, into the delta-V accumulated since the last reset, and publishes it for the burn executor. The first sample after a reset starts the accumulation window and is not integrated, so the accumulated delta-V is the integral over the time elapsed since then. The accelerometer bias is a data dependency, which the state manager publishes from the command that set the burn up, and is subtracted from each sample. Wraps the DvAccumulationAlgorithm C++ algorithm via its C shim.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 4
- **Number of Invokee Connectors** - 2
- **Number of Invoker Connectors** - 2
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - *None*
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - 1
- **Number of Data Dependencies** - 2
- **Number of Packets** - *None*

3.2 Diagram

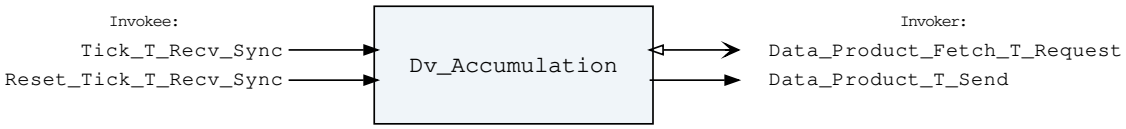


Figure 1: Dv Accumulation component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Dv Accumulation Invokee Connectors

Name	Kind	Type	Return_Type	Count
Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Reset_Tick_T_ Recv_Sync	recv_sync	Tick.T	-	1

Connector Descriptions:

- **Tick_T_Recv_Sync** - Integrate the acceleration of this tick and publish the accumulated delta-V.
- **Reset_Tick_T_Recv_Sync** - Zero the accumulated delta-V and start a new accumulation window at the next tick. The assembly fires this when a burn is set up, before the burn executor is started.

3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Dv Accumulation Invoker Connectors

Name	Kind	Type	Return_Type	Count
Data_Product_ Fetch_T_Request	request	Data_Product_ Fetch.T	Data_Product_ Return.T	1
Data_Product_T_ Send	send	Data_Product.T	-	1

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database.
- **Data_Product_T_Send** - The data product invoker connector

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Dv Accumulation Set Id Bases Parameters

Name	Type
`Data_Product_Id_Base`	`Data_Product_Types.Data_Product_Id_Base`

Parameter Descriptions:

- **`Data_Product_Id_Base`** - The value at which the component's data product identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Dv Accumulation Map Data Dependencies Parameters

Name	Type
`Accel_Average_Id`	`Data_Product_Types.Data_Product_Id`
`Accel_Average_Stale_Limit`	`Ada.Real_Time.Time_Span`
`Accel_Bias_B_Id`	`Data_Product_Types.Data_Product_Id`
`Accel_Bias_B_Stale_Limit`	`Ada.Real_Time.Time_Span`

Parameter Descriptions:

- **`Accel_Average_Id`** - The external data product ID used to fetch the value for the 'Accel_Average' data dependency.
- **`Accel_Average_Stale_Limit`** - The stale limit used for the 'Accel_Average' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.
- **`Accel_Bias_B_Id`** - The external data product ID used to fetch the value for the 'Accel_Bias_B' data dependency.
- **`Accel_Bias_B_Stale_Limit`** - The stale limit used for the 'Accel_Bias_B' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the delta-V accumulation algorithm with the control period. The `init` subprogram requires the following parameters:

Table 5: Dv Accumulation Implementation Initialization Parameters

Name	Type	Default Value
`Control_Period`	`Basic_Types.Positive_Short_Float`	*None provided*

Parameter Descriptions:

- **`Control_Period`** - [s] Time between two algorithm updates, the integration step of each sample, so it must equal the rate at which the component is scheduled. Fixed by the assembly, so it is not a runtime parameter. The type keeps it greater than zero.

3.6 Commands

The Dv Accumulation component has no commands.

3.7 Parameters

The Dv Accumulation component has no parameters.

3.8 Events

The Dv Accumulation component has no events.

3.9 Data Products

Data products for the Dv Accumulation component.

Table 6: Dv Accumulation Data Products

Local ID	Data Product Name	Type
`0x0000 (0)`	`Accumulated_Delta_V`	`Packed_F32x3.T`

Data Product Descriptions:

- **`Accumulated_Delta_V`** - [m/s] Delta-V accumulated in body frame components since the last reset, with the accelerometer bias removed.

3.10 Data Dependencies

Data dependencies for the Dv Accumulation component.

Table 7: Dv Accumulation Data Dependencies

Data Dependency Name	Type
`Accel_Average`	`Packed_F32x3.T`
`Accel_Bias_B`	`Packed_F32x3.T`

Data Dependency Descriptions:

- **`Accel_Average`** - [m/s^2] The majority voted accelerometer average in body frame components, the body frame acceleration that gravity does not cause. The majority vote publishes it every tick, ahead of this component.
- **`Accel_Bias_B`** - [m/s^2] The accelerometer bias in body frame components, the offset the measured acceleration contains, which is subtracted from each sample. The state manager publishes it once, from the command that sets the burn up, and it holds for the whole burn, so a stale value is used like a fresh one.

3.11 Packets

The Dv Accumulation component has no packets.

3.12 Faults

The Dv Accumulation component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Dv_Accumulation_Tests* Test Suite

This is a unit test suite for the Dv Accumulation component

Test Descriptions:

- **`Test`** - Feed a constant acceleration and check the accumulated delta-V grows by the acceleration times the control period on each tick after the first, which starts the window, to ensure the Ada to C to C++ integration is sound.
- **`Test_Bias`** - Ensure the accelerometer bias the state manager publishes is subtracted from each sample.
- **`Test_Stale_Bias`** - Ensure a bias that is stale, since it is published once when the burn is set up, is subtracted from each sample like a fresh one.
- **`Test_Reset`** - Ensure the reset connector zeroes the accumulated delta-V and the next sample starts a new window without being integrated.
- **`Test_Invalid_Data_Dependency`** - Ensure a data dependency with the wrong identifier fails an assertion and nothing is published.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 8: Data_Product Packed Record : 1096 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_ Header.T	-	72	0	71	-
Buffer	Data_Product_ Types.Data_ Product_ Buffer_Type	-	1024	72	1095	Header. Buffer_Length

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 9: Data_Product_Fetch Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	0	15

Field Descriptions:

- **Id** - The data product identifier

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 10: Data_Product_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	48	63
Buffer_Length	Data_Product_ Types.Data_Product_ Buffer_Length_Type	0 to 128	8	64	71

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 11: Data_Product_Return Packed Record : 1104 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
The_Status	Data_Product_Enums.Fetch_Status.E	0 => Success 1 => Not_Available 2 => Id_Out_Of_Range	8	0	7	-
The_Data_Product	Data_Product.T	-	1096	8	1103	-

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 12: Packed_F32x3 Packed Array : 96 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	3	96

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 13: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces.Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces.Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^16) sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 14: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 15: Fetch_Status Literals:

Name	Value	Description
Success	0	The data product was returned successfully.
Not_Available	1	No data product is yet available for the provided id.
Id_Out_Of_Range	2	The data product id was out of range.